

Document number: P0238R0
Date: 2016-02-07
Project: Programming Language C++, Evolution Working Group
Reply-to: Tomasz Kamiński <tomaszkam at gmail dot com>

Return type deduction and SFINAE

Introduction

This paper proposes to make return type deduction failure a SFINAEable error instead of a hard error.

Motivation and Scope

For an example, let's consider the following implementation of `variadic_negator`:

```
template<typename F>
class variadic_negator_functor
{
    F f_;

public:
    explicit variadic_negator_functor(F a_f) : f_(a_f) {}

    template<typename... Args>
    auto operator()(Args&&... args)
        -> decltype(!this->f_(std::forward<Args>(args)...))
    { return !this->f_(std::forward<Args>(args)...); }
};

template<typename F>
variadic_negator_functor<F> variadic_negator(F f)
{
    return variadic_negator_functor<F>(std::move(f));
}
```

In the implementation we can see an obvious repetition of the function invocation that is placed both in the return type declaration and in the return statement. We could avoid this necessary repetition by using return type deduction for the normal function and declare return type as `decltype(auto)`. As consequence of this change we would affect the overload resolution for calls of the `operator()`: in case of `decltype(expr)` this candidate was not considered to be viable in case when the `f` was is not callable with `args...` or result of the invocation cannot be negated, while in case of use of `decltype(auto)` this candidate is always viable.

In the context of the discussed class for every invocation `variadic_negator(f)(args...)` there is only one function candidate, and at first glance we may consider this change to be acceptable, as it will only change the error messages caused by invalid invocations of the functor. However, such reasoning omits situations when the availability of other functions depends on the validity of the call to `variadic_negator`. As an example, we may consider the following:

```
template<typename F>
auto invoke(F f) -> decltype(f(0)); //1

template<typename F>
auto invoke(F f) -> decltype(f(std::string())); //2

bool test(int);
```

For above declarations the expression `invoke(&test)` is valid and selects the first overload, the same applies for `invoke(variadic_negator(&test))` if `decltype(expr)` is used as return type. However in case of `decltype(auto)` implementation the code will emit hard error about lack of conversion from `std::string` to `int`.

To understand this situation we need to realize that expression in the form `decltype(foo(args...))` requires compiler to determine return type of selected overload of `foo` which in case of function template that uses return type deduction leads to its instantiation and according to current wording, any error occurring in such instantiation is considered to be hard-error. In case of our example the expression `invoke(variadic_negator(&test))` leads to instantiation of `operator()` with `std::string&&`, that is invalid as `&test` accepts only integers.

This paper proposes to address above issue by requiring that any error occurring during the instantiation of function template triggered by return type deduction in unevaluated context should lead to substitution failure. As a result, programmers will be

allowed to check the validity of the call to any function even if it happens to use the return type deduction feature.

Functions accepting `std::function` as parameter

It may be pointed out that presented example of the `invoke` function is artificial and such situations will not normally occur in the code. However as result of resolution of [LWG issue #2132](#) similar mechanism is used to eliminate `std::function` constructor in case when object is not callable. As consequence same problem would occur in case when `invoke` would be defined as:

```
bool invoke(std::function<bool(int)> f);           //1
bool invoke(std::function<bool(std::string)> f); //2
```

Interoperation with lambda functions

Problem that is intended to be addressed by this paper occurs only in situations when functor with template `operator()` that uses return type deduction is passed to the function. As writing such classes is rarely involved in day-to-day programming, its impact may be neglected as affecting only experts, that should be aware of above SFINAE nuances and design their code accordingly.

However with generic lambdas in C++14 provides we have introduced short syntax for declaration of such classes, for example the expression `[] (auto x) { return x == 2; }` creates an unnamed class with template `operator()` that uses `auto` return type deduction. As consequence programmers are exposed to quirks of design of return type deduction even for simple code.

If we consider following declarations:

```
bool invoke(std::function<bool(int)> f);           //1
bool invoke(std::function<bool(std::string)> f); //2
```

The expression in the form `[] (int x) { return x == 2; }` is well formed and selects first overload, while seemingly identical expression that uses new generic lambda `[] (auto x) { return x == 2; }` produces hard error.

In the light of advancement of the development of concept for C++ language, it may be argued that such problem will not exist, as instead of use of unconstrained (`auto`) template argument user will use short concept syntax. Considering the fact that main motivation behind introduction lambda expression was simplification of creation of ad-hoc functors that are used locally, then we can expect that in such cases lambda will be only constrained by ad-hoc requirements. However in contrast to lambda expression, such constraints cannot be declared locally and pursue to constrain every generic lambda in the code will lead to creation of small syntactic constraints (like `has_operator_plus<T, U>`) mimicking failed C++11 concept design.

Furthermore it is worth noticing that in above example created closure is not used in polymorphic way and we could use `int` parameter. However we may still argue that use of `auto` lambda parameter leads to better code, in the same way as use of `auto` instead of specific type in variable declaration (no unintended conversion, better maintainability).

Error messages with Concepts

Return type deduction (and function instantiation) may be also triggered as part of the constraint check and also in this case any failure occurring during this instantiation will lead to hard error and unreadable compiler message produced by compiler.

For example in case of following declarations:

```
template<typename F, typename... T>
concept bool Predicate = requires(F f, T... t) {
    { f(t...) } -> bool;
};

bool invoke(Predicate<std::string>);
bool is_even(int x) { return x == 2; }
```

The expression `invoke(&is_even)` produces short and concise message on GCC 6.0 branch:

```
prog.cc: In function 'int main()':
prog.cc:17:19: error: cannot call function 'bool invoke(auto:1) [with auto:1 = bool (*)(int)]'
    invoke(is_even);
              ^
prog.cc:12:6: note:   constraints not satisfied
    bool invoke(Predicate<std::string>);
    ~~~~~
prog.cc:12:6: note:   concept 'Predicate<bool (*)(int), std::__cxx11::string>' was not satisfied
```

However equivalent code written using generic lambda `invoke([] (auto x) { return x == 2; })` produces 795 lines of error message.

In addition similar error would be produced for invocation of STL algorithm, e.g. `std::all_of(begin(v), end(b), [] (auto x) {`

`return x == 2; }` with `v` being declared as `std::vector<std::string>`.

Design Decisions

This paper proposes that any error occurring in function template instantiation required be return type deduction triggered in unevaluated context should lead to substitution failure instead of hard error.

Overloading on function body

During overload resolution for the call to the function `foo` no return type deduction is performed for the candidate declarations of the function `foo` that uses return type deduction.

For example in the case of the following declarations:

```
template<typename F>
auto apply(F f, int i) { return f(i); }           // 1

template<typename F>
auto apply(F f, std::string s) { return f(s); } // 2
```

Overload resolution for the invocation in the form `apply([](int x) { return x; }, std::string())` will still select second candidate and hard error will be produced by its instantiation. In addition the following two declarations will still lead to double definition error:

```
template<typename T>
auto foo(T t) { return t == 0; }

template<typename T>
auto foo(T t) { return t == "0"; }
```

To summarize this paper does not require the compiler to eliminate the function overload based of validity of their body in case when return type deduction is used and, as consequence, does not impose need to support mangling of whole function body into signature.

Introducing new overloads

In some situations the user may still want to eliminate function overload based on the whole function body and changed proposed in this paper support such use cases by introducing additional level of indirection. For example following function:

```
template<typename T>
decltype(auto) foo(T&& t) { ... }
```

Can be transformed into:

```
template<typename T>
decltype(auto) foo_impl(T&& t) { ... }

template<typename T>
auto foo(T&& t) -> decltype(foo_impl(std::forward<T>(t)))
{ return foo_impl(std::forward<T>(t)); }
```

Example use case of such feature would be implementation of the binary `overload` function (as proposed in [P0051R1: C++ generic overload function \(Revision 1\)](#)).

If we consider implementation presented in the paper:

```
template<class F1, class F2> struct overloaded : F1, F2
{
    overloaded(F1 x1, F2 x2) : F1(x1), F2(x2) {}
    using F1::operator();
    using F2::operator();
};

template<class F1, class F2>
overloaded<F1, F2> overload(F1 f1, F2 f2)
{ return overloaded<F1, F2>(f1, f2); }
```

For this implementation every call to the functor `f = overload([](auto t) { return t == 0; }, [](auto t) { return t.empty(); })` will be lead to ambiguity error, as each lambda can accept any movable type. However with the proposed change and slight change of the `overloaded` code, we would be able to make such call unambiguous unless passed object is both comparable with `0` and have `empty()` method.

```
template<class F1, class F2> struct overloaded : F1, F2
{
    overloaded(F1 x1, F2 x2) : F1(x1), F2(x2) {}

    template<typename... Args>
    auto operator()(Args&&... args)
        -> decltype(this->F1::operator()(std::forward<Args>(args)...))
    { return this->F1::operator()(std::forward<Args>(args)...); }

    template<typename... Args>
    auto operator()(Args&&... args)
        -> decltype(this->F2::operator()(std::forward<Args>(args)...))
    { return this->F2::operator()(std::forward<Args>(args)...); }
};
```

Impact on build times

The increase of build times caused by additional failed function template instantiation may be considered as the main argument against introduction of proposed change. This section discuss in detail scope of potential impact compared to alternatives.

Firstly lets consider the impact on build time of existing valid programs. By definition every return type deduction preformed in such program was successful (otherwise hard-error would be emitted and program would be invalid), so no additional instantiations will be introduced. As consequence build time of existing code should not be affected.

Secondly lets out consider the program for which one of return type deduction failed. If failed instantiation was not triggered in unevaluated context, such program will remain ill-formed. Lets assume that above deduction failure occurred during overload resolution from the set of candidate that contains N function using return type deduction, out of which K would be instantiated without any error. According the current rules the hard error will be emitted during the first failed deduction, so depending on the implementation program may already perform between 1 (1st checked candidate failed) and $K + 1$ (all well-formed candidates were checked before failure) template instantiations. In case of the proposed change up-to N instantiations may be performed.

As showed above, acceptance of this change will increase the limit of potential functions template instantiations and by consequence increase compilation time. However we should also consider the cost of currently existing alternative, that will make such program well-formed. The simplest approach would be to change the definition of existing function to use `decltype(expr)`, where `expr` would be valid if the function body would be well formed. Such transformation may be already done by transforming statement into coma-separated list of expression (local lambda should be transformed to functions object declared outside of function scope). It is not unreasonable to assume that the cost of checking validity of such expression would be comparable to potential cost of function body instantiations. In addition the use of `decltype(expr)` approach incurs additional cost of mangling of function signature.

Impact On The Standard

This proposal has no dependencies beyond a C++14 standard.

Nothing depends on this proposal.

Proposed wording

The proposed wording changes refer to [N4567](#) (C++ Working Draft, 2015-11-09).

Change the paragraph 7.1.6.4 auto specifier [dcl.spec.auto] p8.

Return type deduction for a function template with a placeholder in its declared type occurs when the definition is instantiated even if the function body contains a `return` statement with a non-type-dependent operand. [*Note*: Therefore, any use of a specialization of the function template will cause an implicit instantiation. **Any errors that arise from this instantiation are not in the immediate context of the function type and can result in the program being ill-formed.** — *end note*] If this instantiation occurs as part of template argument substitution, any errors that arise from it should lead to deduction failure ([temp.deduct] 14.8.2).

Implementability

This paper requires compiler implementers to be able to turn instantiation error from function body into substitution error. As consequence the compiler vendors would be required to implement the ability to backtrack from the errors occurring in

expression that are not allowed in unevaluated context (like lambda expression), however no mangling support for this expression is required.

Acknowledgements

Ville Voutilainen and Andrzej Krzemiński offered many useful suggestions and corrections to the proposal.

References

1. Marshall Clow, "C++ Standard Library Defect Report List (Revision R93)" (N4485, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4485.html>)
2. Vicente J. Botet Escriba, "C++ generic overload function (Revision 1)" (P0051R1, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/p0051r1.pdf>)
3. Richard Smith, "Working Draft, Standard for Programming Language C++" (N4567, <http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2015/n4567.pdf>)